TESTNG

Class 1 - Complete Notes

From first @Test to practical Selenium test execution

Concepts | Code walkthroughs | Real-world scenarios | Best practices

Prepared from the supplied "TestNG class 1" notes, expanded with practical examples.

> **Important security note** The original sample contains a hard-coded email and password. They are intentionally not repeated here. Use test-only accounts and externalize credentials.

How to use these notes

Start with Sections 1-5 to understand the framework and execute your first test. Sections 6-12 explain the annotations from the class notes. Sections 13-18 show how the concepts fit into a maintainable Selenium test suite. The final sections provide exercises, troubleshooting, and a quick revision sheet.

Contents

- 1. What is TestNG?
- 2. Project setup: Maven and Eclipse
- 3. Your first TestNG test
- 4. Understanding @Test and test execution order
- 5. Priority: what it does and what it does not do
- 6. Repeating tests: invocationCount, invocationTimeOut, threadPoolSize
- 7. Timeout: timeOut
- 8. Skipping and disabling tests
- 9. Assertions: deciding pass or fail
- 10. Preconditions and postconditions: lifecycle annotations
- 11. Data-driven testing with @DataProvider
- 12. Groups and dependencies
- 13. A safer Selenium + TestNG example
- 14. Parallel execution
- 15. Listeners and reporting
- 16. Re-running failed tests and handling exceptions
- 17. Common mistakes and debugging checklist
- 18. Hands-on practice exercises
- 19. Interview revision sheet
- 20. Glossary and references

> **Learning principle** A test is not successful merely because its code ran without throwing an exception. A reliable test verifies an expected outcome with assertions and cleans up what it created.

1. What is TestNG?

TestNG stands for “Test Next Generation.” It is an open-source testing framework for Java. It helps organize, configure, run, and report test methods. Selenium controls the browser; TestNG controls the test execution around that browser automation.

Tool / concept	Main responsibility	Example
Selenium WebDriver	Interact with a browser and web page elements.	Enter email, click Login, read a page heading.
TestNG	Discover and execute tests, arrange setup/cleanup, group tests, repeat or parameterize them, and report results.	Run login tests, mark assertions, rerun failures.
Maven	Manage Java dependencies and build/test commands.	Download TestNG and Selenium libraries.
Assertion	Compare actual behavior with expected behavior.	Verify that the dashboard heading says “Dashboard”.

What can TestNG do?

- Create test methods using @Test.
- Group related tests, such as smoke, regression, checkout, or API tests.
- Set execution priority when an ordering rule is genuinely required.
- Pass multiple data sets into one test using @DataProvider or XML parameters.
- Repeat one method with invocationCount and control invocation time.
- Apply test time limits with timeOut.
- Disable selected tests, mark expected exceptions, and make assertions.
- Run setup and cleanup methods with lifecycle annotations.
- Run tests in parallel when the tests and test resources are thread-safe.
- Use listeners for logging, screenshots, custom reporting, or failure hooks.
- Generate TestNG reports and rerun selected failed tests.

> **Beginner mental model** TestNG is the test manager. Your @Test method is the actual test, Selenium performs browser actions, and assertions decide whether the observed result matches the requirement.

Typical execution flow

```
Maven / IDE starts TestNG
      |
      v
TestNG discovers @Test methods and configuration methods
      |
      v
@Before... methods prepare required resources
      |
      v
@Test methods execute Selenium / API / Java assertions
      |
      v
@After... methods clean up resources
      |
      v
TestNG records PASS / FAIL / SKIP and creates reports
```

2. Project setup: Maven and Eclipse

The supplied class notes mention installing the TestNG plug-in from Eclipse Marketplace and adding a dependency to pom.xml. The plug-in helps run TestNG from Eclipse, while the Maven dependency makes TestNG available to the project. They solve related but different problems.

Step A - Create or open a Maven project

- Confirm Java is configured in the IDE and that the project has a pom.xml.
- Create test classes under src/test/java. Keep application code under src/main/java when the project contains production code too.
- In Eclipse, install the TestNG plug-in via Help > Eclipse Marketplace and search for TestNG, or use the official TestNG Eclipse update site.

Step B - Add Maven dependencies

Use a TestNG version compatible with the project JDK and other libraries. The TestNG project release page lists version 7.12.0 as the latest release surfaced during preparation (17 October 2026); verify the official release page before upgrading. TestNG 7.12.0 requires Java 11 or newer according to the project repository.

Example pom.xml dependencies

```
<dependencies>
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.12.0</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>REPLACE_WITH_VALIDATED_VERSION</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

> **Version note** The Selenium version is deliberately left as a placeholder. Pin a real version that your project has validated; do not copy an empty or placeholder version into a working pom.xml.

Step C - Run the test

```
# Run tests with Maven from the project directory
mvn test

# In Eclipse: right-click a TestNG class > Run As > TestNG Test
# Or run a test suite XML file as a TestNG Suite
```

A typical project structure looks like this:

```
my-automation-project/
 pom.xml
 src/
  test/
   java/
    tests/
     LoginTests.java
    pages/
     LoginPage.java
   resources/
    testng.xml
```

3. Your first TestNG test

A Java method becomes a TestNG test method when it is annotated with @Test (or when a class-level @Test configuration makes its methods tests). In the simplest beginner example, put @Test directly above a public method.

```
package tests;

import org.testng.annotations.Test;

public class FirstTest {

   @Test
   public void verifyApplicationTitle() {
      System.out.println("This is my first TestNG test");
   }

   public void helperMethod() {
      System.out.println("This is a normal Java method");
   }
}
```

Explanation:

- @Test tells TestNG to execute verifyApplicationTitle as a test method.
- The class does not need a main method when TestNG is the test runner.
- helperMethod has no @Test annotation, so TestNG does not treat it as a test method in this example. It can still be called by another method as an ordinary Java method.
- System.out.println demonstrates execution only. A real test should assert an expected result instead of only printing a message.

A first test with an assertion

```
import org.testng.Assert;
import org.testng.annotations.Test;

public class SimpleAssertionTest {
   @Test
   public void verifyCalculation() {
      int actual = 2 + 3;
      int expected = 5;
      Assert.assertEquals(actual, expected, "The sum should be five");
   }
}
```

If the actual result equals the expected result, the assertion passes. If not, TestNG records the test as failed and reports the assertion message.

4. Understanding @Test and test execution order

The supplied notes say that TestNG executes tests alphabetically by default and that priority controls execution order. This is a useful first observation, but do not design a reliable suite around assumed implicit order. TestNG also considers dependencies, groups, suite configuration, and parallel settings. Explicitly model a required dependency or, preferably, make tests independent.

Three annotated test methods

```
import org.testng.annotations.Test;

public class ContactTests {
    @Test
    public void login() {
        System.out.println("Login test");
    }

    @Test
    public void createNewContact() {
        System.out.println("Create contact test");
    }

    @Test
    public void deleteContact() {
        System.out.println("Delete contact test");
    }
}
```

All three methods are test methods. This code does not guarantee a business workflow in which login must occur once before creation and deletion. A robust suite normally logs in during setup for each test or manages authentication through a fixture/API. A deletion test should arrange its own contact data and should not require another test to have run first.

5. Priority: what it does and what it does not do

```
@Test(priority = 1)
public void loginTest() { }

@Test(priority = 2)
public void createContactTest() { }

@Test(priority = 3)
public void deleteContactTest() { }
```

Rule	Meaning
Lower number first	A smaller priority value is scheduled before a larger one; for example, -2 is before 0, and 0 is before 1.
Default value	The default priority is 0.
Negative values	Negative priorities are valid.
Same priority	When methods have equal priority, TestNG uses its method-ordering rules; examples commonly appear alphabetically. Do not rely on this as a business-workflow contract.
No priority	When order does not matter, omit priority. Prefer independent tests over making test success depend on test-method ordering.

> **Better than priority for a hard dependency** If test B truly cannot run without test A, TestNG supports dependsOnMethods or dependsOnGroups. However, for UI regression tests, independent setup is usually more reliable than chaining test cases together.

Dependency example - use only when intentional

```
@Test
public void loginTest() {
    // arrange and verify successful login
}

@Test(dependsOnMethods = "loginTest")
public void accountPageTest() {
    // runs after loginTest; if the dependency fails, this can be skipped
}
```

6. Repeating tests: invocationCount, invocationTimeOut, threadPoolSize

These attributes are all attached to @Test. They are not interchangeable: invocationCount is a repeat count, invocationTimeOut is an overall time limit for those invocations, and threadPoolSize controls parallel invocation threads when invocationCount is specified.

6.1 invocationCount

```
@Test(invocationCount = 5)
public void verifySearchEndpoint() {
    // This test method is invoked five times.
}
```

Use cases: check a stable API several times, repeat a small reliability test, or quickly observe intermittent failures. Repetition alone is not a performance test and does not prove production reliability.

6.2 invocationTimeOut

```
@Test(invocationCount = 5, invocationTimeOut = 15000)
public void verifyRepeatedOperation() {
    // TestNG limits the total time for the invocation set.
}
```

The attribute spelling is invocationTimeOut (capital O in Out) in TestNG. It sets the maximum time in milliseconds for the cumulative invocations and is ignored when invocationCount is not set.

6.3 threadPoolSize

```
@Test(invocationCount = 10, threadPoolSize = 3, timeOut = 10000)
public void checkService() {
    // Up to three invocation threads can be used for these invocations.
}
```

In this example, TestNG invokes the method ten times using a pool of three threads. Each invocation may overlap with other invocations, so test data and resources must be isolated. Do not share one mutable Selenium WebDriver instance across these parallel invocations.

Attribute	Question it answers	Unit / key point
invocationCount	How many times should the test method run?	Count; default is 1.
invocationTimeOut	How long may the full set of repeated invocations take?	Milliseconds; requires invocationCount.
threadPoolSize	How many concurrent threads can execute the repeated invocations?	Thread count; requires invocationCount.
timeOut	How long may one test invocation take?	Milliseconds.

Real-world scenario A search smoke test is repeated across a few known queries to catch intermittent instability. Each run uses unique or read-only data, the expected result is asserted, and logs identify which query failed.

7. Timeout: timeOut

```
@Test(timeOut = 3000)
public void responseShouldArriveQuickly() {
   // The test must finish within 3,000 milliseconds (3 seconds).
}
```

TestNG timeOut is a test-level execution guard. It does not mean the application has become fast or that all browser actions will automatically wait correctly. When the limit is exceeded, TestNG reports a timeout failure.

TestNG timeout vs Selenium wait

Mechanism	Purpose	Example
TestNG timeOut	Limit the duration of a test invocation.	@Test(timeOut = 5000)
Selenium WebDriverWait	Wait for a specific browser condition, such as a visible button or a title.	wait.until(visibilityOfElementLocated(...))
Thread.sleep	Pause the current thread for a fixed duration.	Thread.sleep(2000)

Best practice Prefer explicit waits for the condition you need. Thread.sleep adds an unconditional delay and can make tests slow or flaky. Use a TestNG timeout only as a safety boundary, not as a replacement for condition-based waits.

8. Skipping and disabling tests

A skipped test is not the same as a passed test. It means the test did not execute normally. A skipped result should be understood, explained, and monitored.

Preferred explicit switch

```
import org.testng.annotations.Test;

public class FeatureTests {
   @Test(enabled = false)
   public void unfinishedFeatureTest() {
      // TestNG will not execute this test while enabled=false.
   }
}
```

The supplied notes also mention @Ignore. TestNG provides @Ignore to ignore test methods or classes, depending on where it is applied. For a single method, @Test(enabled = false) is often easier to see and control. Confirm your TestNG version and scope before using @Ignore.

> **Do not hide failures** Do not disable a test simply because it is failing. Record the known defect, owner, and reason; ideally manage quarantine with a visible tag or issue reference and restore the test when fixed.

9. Assertions: deciding pass or fail

An assertion compares expected behavior with actual behavior. Without assertions, a method may execute clicks and still pass even if the application displays the wrong page or saves the wrong data.

```
import org.testng.Assert;
import org.testng.annotations.Test;

public class AssertionsExample {
   @Test
   public void assertExamples() {
      String actualTitle = "Dashboard";
      Assert.assertEquals(actualTitle, "Dashboard", "Unexpected page title");
      Assert.assertTrue(actualTitle.contains("Dash"), "Title should contain Dash");
      Assert.assertFalse(actualTitle.isBlank(), "Title must not be blank");
   }
}
```

Hard assertions and SoftAssert

- Hard assertion: methods such as Assert.assertEquals fail immediately at the failing assertion; remaining statements in that test method do not execute normally.
- Soft assertion: collects multiple assertion failures and reports them together when assertAll() is called.

Soft assertion example

```
import org.testng.annotations.Test;
import org.testng.asserts.SoftAssert;

public class ProductDetailsTest {
   @Test
   public void verifyProductDetails() {
      SoftAssert soft = new SoftAssert();
      soft.assertEquals("Wireless Mouse", "Wireless Mouse", "Product name mismatch");
      soft.assertEquals("₹999", "₹999", "Price mismatch");
      soft.assertTrue(true, "Product should be available");
      soft.assertAll(); // Required: reports any collected failures.
   }
}
```

Important For every SoftAssert test, call assertAll() at the end. Otherwise collected failures may not fail the test, creating a false green result.

10. Preconditions and postconditions: lifecycle annotations

TestNG configuration annotations run before or after scopes of tests. They are useful for setting up a browser, logging in, creating fixtures, collecting diagnostics, and closing resources.

Annotation	When it runs	Typical use
@BeforeSuite	Once before all tests in the suite.	Global test-environment validation.
@AfterSuite	Once after all tests in the suite.	Close suite-wide resources.
@BeforeTest	Before test classes under a <test> block in testng.xml.	Configure a group of classes for a suite XML test block.
@AfterTest	After the related <test> block completes.	Clean up resources scoped to that test block.
@BeforeClass	Before the first test method in the current class.	Class-level setup.
@AfterClass	After all test methods in the current class.	Class-level cleanup.
@BeforeMethod	Before each @Test method invocation.	Create a fresh browser or prepare test data.
@AfterMethod	After each @Test method invocation.	Capture diagnostics and quit the browser.
@BeforeGroups	Before the first method in specified groups.	Prepare data for a group.
@AfterGroups	After the last method in specified groups.	Clean up data for a group.

Lifecycle setup and teardown example

```
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

public class LifecycleDemo {
    @BeforeMethod
    public void setUp() {
        System.out.println("Prepare test preconditions");
    }

    @Test
    public void testA() {
        System.out.println("Execute test A");
    }

    @Test
    public void testB() {
        System.out.println("Execute test B");
    }

    @AfterMethod(alwaysRun = true)
    public void tearDown() {
        System.out.println("Clean up after the test");
    }
}
```

The method-level configuration in this example runs before and after each test invocation. `alwaysRun = true` on teardown helps cleanup run even when a test fails or is skipped due to a dependency. Teardown should tolerate partially initialized resources.

11. Data-driven testing with @DataProvider

Data-driven testing means running the same test logic with multiple inputs. Instead of duplicating nearly identical methods for each username or search term, keep test logic in one method and provide rows of data.

DataProvider example - placeholder UI result is clearly marked

```
import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

public class LoginValidationTest {

  @DataProvider(name = "invalidLoginData")
  public Object[][] invalidLoginData() {
    return new Object[][] {
      {"", "validPassword", "Email is required"},
      {"user@example.test", "", "Password is required"},
      {"not-an-email", "validPassword", "Enter a valid email"}
    };
  }

  @Test(dataProvider = "invalidLoginData")
  public void invalidLoginShowsValidation(String email, String password, String expectedMessage) {
    // Arrange: open the login page.
    // Act: enter email/password and submit.
    String actualMessage = expectedMessage; // Placeholder: read the real UI message here.
    // Assert: replace the placeholder with the actual web element text.
    Assert.assertEquals(actualMessage, expectedMessage);
  }
}
```

Each Object[] row corresponds to one invocation of the test method. Its values are passed positionally to method arguments. The test method parameter types and count must match the data row.

> **Do not mistake the sample assertion for a real UI check** The example uses a placeholder variable to teach parameter mapping without pretending to connect to an application. A real test must read the validation message from the page and assert that text.

External parameters through testng.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.1.dtd">
<suite name="Login suite">
 <test name="Chrome smoke">
  <parameter name="baseUrl" value="https://your-test-app.example"/>
  <classes>
   <class name="tests.LoginXmlParameterTest"/>
  </classes>
 </test>
</suite>
```

```
import org.testng.annotations.Parameters;
import org.testng.annotations.Test;

public class LoginXmlParameterTest {
  @Test
  @Parameters({"baseUrl"})
  public void openApplication(String baseUrl) {
    System.out.println("Target environment: " + baseUrl);
  }
}
```

Use DataProvider for several rows of inputs. Use XML parameters when selecting configuration values such as an environment URL, browser mode, or test-user alias. Avoid putting passwords or tokens directly in suite XML committed to source control.

12. Groups and dependencies

Groups let a team choose which kinds of tests to run. Typical groups include smoke, regression, sanity, integration, and destructive. A test can belong to more than one group.

```
@Test(groups = {"smoke", "login"})
public void validUserCanLogin() { }

@Test(groups = {"regression", "contacts"})
public void userCanCreateContact() { }
```

Select a group in testng.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.1.dtd">
<suite name="Regression Suite">
 <test name="Smoke only">
  <groups>
   <run>
    <include name="smoke"/>
   </run>
  </groups>
  <packages>
   <package name="tests"/>
  </packages>
 </test>
</suite>
```

Dependencies (dependsOnMethods or dependsOnGroups) express that one test requires another to succeed. If the prerequisite fails, the dependent test may be skipped. Use dependencies sparingly: a long dependency chain can make failures hard to diagnose and reduce the value of independent tests.

13. A safer Selenium + TestNG example

The supplied FreeCRM example launches Chrome, navigates to a login page, enters credentials, and then adds a contact. It also uses Thread.sleep, puts WebDriver in a shared field, and does not close the browser. The sample below demonstrates a safer structure. Replace the URL and locators with values from your own test application.

> **Security** Do not paste production credentials into automation code. Create a dedicated non-production test account and load secrets from environment variables or a secret manager.

End-to-end structure - update URL, selectors, and success condition

```
package tests;
import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

public class ContactCreationTest {
    private WebDriver driver;
    private WebDriverWait wait;

    @BeforeMethod
    public void setUp() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
        driver.get(System.getProperty("app.baseUrl", "https://your-test-app.example"));
    }

    @Test
    public void createContactShowsSuccess() {
        String email = System.getenv("TEST_USER_EMAIL");
        String password = System.getenv("TEST_USER_PASSWORD");
        Assert.assertNotNull(email, "Set TEST_USER_EMAIL before running the test");
        Assert.assertNotNull(password, "Set TEST_USER_PASSWORD before running the test");
        driver.findElement(By.id("email")).sendKeys(email);
        driver.findElement(By.id("password")).sendKeys(password);
        driver.findElement(By.xpath("//button[normalize-space()='Login']")).click();
        wait.until(ExpectedConditions.visibilityOfElementLocated(By.xpath("//*[normalize-space()='Contacts']"))).click();
        wait.until(ExpectedConditions.elementToBeClickable(By.xpath("//button[normalize-space()='Create']"))).click();
        driver.findElement(By.id("first-name")).sendKeys("Automation");
        driver.findElement(By.id("last-name")).sendKeys("Test");
        driver.findElement(By.xpath("//button[normalize-space()='Save']")).click();
        // Replace with a stable success message or persisted record assertion.
        boolean contactVisible = wait.until(
            ExpectedConditions.visibilityOfElementLocated(
                By.cssSelector("[data-testid='contact-success']")))
            .isDisplayed();
        Assert.assertTrue(contactVisible, "Created contact should be visible");
    }

    @AfterMethod(alwaysRun = true)
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

Code walkthrough:

1. `@BeforeMethod` creates a fresh browser and opens the configured base URL before each test method.
2. `WebDriverWait` waits for a specific UI condition instead of sleeping for a guessed duration.
3. Credentials are read from environment variables, not hard-coded into the Java source.
4. The test performs the user action and then asserts a visible outcome. Verify the exact locator and success condition against your actual DOM; the sample app and locators are intentionally generic.
5. `@AfterMethod(alwaysRun = true)` calls `driver.quit()` to close the browser even after many test failures.

> **Framework-quality follow-up** In a larger suite, move login/contact actions to Page Object classes, avoid brittle text-heavy XPath when stable IDs or test attributes exist, create unique test data, and delete created records in a cleanup step or API fixture.

14. Parallel execution

Parallel testing runs multiple tests or invocations at the same time to reduce elapsed execution time. Parallelization is not a free speed boost: shared accounts, shared files, static variables, a common WebDriver, and reused records can cause data races and flaky failures.

Suite-level parallelization example

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.1.dtd">
<suite name="Parallel smoke suite" parallel="methods" thread-count="3">
 <test name="Web tests">
  <classes>
   <class name="tests.LoginTests"/>
   <class name="tests.ContactTests"/>
  </classes>
 </test>
</suite>
```

Common modes include parallel=methods, parallel=classes, parallel=tests, or parallel=instances. Choose the level that matches resource isolation. Validate your TestNG version and suite structure, and start with a small thread count.

Risk	Mitigation
One shared WebDriver used by multiple tests	Use a separate WebDriver per test thread or a carefully designed ThreadLocal driver manager.
Tests create records with the same name	Generate unique test data, such as a timestamp or UUID.
Tests change the same user/cart/account	Use separate accounts or reset state before each test.
Too many browser processes	Limit thread-count to the resources the CI agent can sustain.
Order-dependent test cases	Make tests independent; avoid parallelizing fragile dependencies.

15. Listeners and reporting

A TestNG listener receives callbacks about the test lifecycle. For example, ITestListener can react to test start, success, failure, or skip. Common uses include structured logs, screenshots on failure, attaching diagnostic metadata, and integrating with a report system.

Basic listener

```
package listeners;

import org.testng.ITestListener;
import org.testng.ITestResult;

public class ExecutionListener implements ITestListener {
   @Override
   public void onTestStart(ITestResult result) {
      System.out.println("START: " + result.getName());
   }

   @Override
   public void onTestSuccess(ITestResult result) {
      System.out.println("PASS: " + result.getName());
   }

   @Override
   public void onTestFailure(ITestResult result) {
      System.out.println("FAIL: " + result.getName());
      System.out.println("Reason: " + result.getThrowable());
      // A real framework may capture a screenshot from the driver for this test.
   }

   @Override
   public void onTestSkipped(ITestResult result) {
      System.out.println("SKIP: " + result.getName());
   }
}
```

Register the listener in testng.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.1.dtd">
<suite name="Listener suite">
 <listeners>
  <listener class-name="listeners.ExecutionListener"/>
 </listeners>
 <test name="Smoke">
  <classes>
   <class name="tests.LoginTests"/>
  </classes>
 </test>
</suite>
```

TestNG produces default reports after execution. Depending on how the suite is launched, look in the configured output directory (commonly test-output) for an HTML report and XML result files. Open the generated index.html to inspect result counts and failures. Build pipelines may publish the report directory as an artifact.

> **Listener best practice** Do not let reporting logic hide or change the test outcome. Avoid printing secrets or personal data in logs. If the listener captures screenshots, use a thread-safe way to locate the WebDriver for the failed test.

16. Re-running failed tests and handling exceptions

Re-run failed tests

TestNG can generate a `testng-failed.xml` file in its output directory for rerunning failed test methods from a suite. This is useful for diagnosis, but a failed test that passes on rerun is still a possible flaky test and should be investigated.

```
# Example: run the failed-test suite XML with TestNG from your IDE.
# In Eclipse: right-click testng-failed.xml > Run As > TestNG Suite
# With a Maven/TestNG setup, configure execution to target that suite XML as appropriate.
```

Expected exceptions

Sometimes a test is specifically verifying that invalid input is rejected by throwing a particular exception. Use expectedExceptions only when the exception itself is the intended behavior. Do not use it to make unexpected application failures appear normal.

```
import org.testng.annotations.Test;

public class ValidationTest {
   @Test(expectedExceptions = IllegalArgumentException.class)
   public void negativeQuantityIsRejected() {
      validateQuantity(-1); // This method should throw IllegalArgumentException.
   }

   private void validateQuantity(int quantity) {
      if (quantity < 0) {
         throw new IllegalArgumentException("Quantity cannot be negative");
      }
   }
}
```

> **“Ignore Exception” correction** A robust test should not swallow exceptions with empty catch blocks. Either let an unexpected exception fail the test, assert that the expected exception is thrown, or handle a known recoverable condition explicitly.

17. Common mistakes and debugging checklist

Mistake	Why it causes trouble	Better approach
Using Thread.sleep everywhere	Tests become slow and can still fail when the app is slower than the fixed delay.	Use WebDriverWait for a specific condition.
Hard-coded credentials	Secrets can leak through Git, reports, or screenshots.	Use environment variables or a secret manager; rotate exposed credentials.
No assertion after an action	A test can pass without verifying the requirement.	Assert a visible, persisted, or API-confirmed outcome.
One WebDriver field reused unsafely	State can leak between tests; parallel runs may collide.	Use per-test setup and teardown; isolate per-thread resources.
No driver.quit()	Browser processes accumulate and slow down local/CI execution.	Always close the driver in teardown.

Mistake	Why it causes trouble	Better approach
Priority used to model all workflow	Tests become order-dependent and fragile.	Arrange data independently; use explicit dependencies only where justified.
SoftAssert without assertAll()	Collected assertion failures may not fail the test.	Call assertAll() after all soft assertions.
Timeout confused with waiting	A test limit does not wait for the exact element or condition.	Use condition-based Selenium waits plus a sensible TestNG timeout.
Blindly rerunning failures	A flaky issue may be hidden rather than fixed.	Track initial failures and investigate nondeterminism.
Disabling a failing test indefinitely	Regression gaps become invisible.	Track the defect/owner and review quarantined tests.

Debugging checklist when a test fails

- Read the first meaningful exception and its stack trace, not only the final “test failed” label.
- Check whether the application URL, environment, test account, and test data are correct.
- Confirm the locator still matches the current DOM and is unique enough for the target element.
- Use an explicit wait for visibility, clickability, URL, or another observable condition.
- Check whether setup failed before the test method started.
- Confirm teardown closes resources and that a previous test did not contaminate the next test.
- For parallel execution, check shared accounts, names, browser instances, and mutable static state.
- Keep a failure screenshot, timestamp, browser logs, and safe diagnostic output when relevant.

18. Hands-on practice exercises

Complete these exercises in a small Maven project. Try each one before reading the expected outcome.

Exercise 1 - First test: Create two @Test methods and one ordinary helper method. Run the class and observe which methods TestNG reports.

Exercise 2 - Assertion: Write a test that verifies 10 / 2 equals 5. Change the expected value to 4 and inspect the failure report.

Exercise 3 - Priority: Create tests with priorities -1, 0, and 2. Run them and observe the ordering. Then remove priorities and confirm why implicit order should not drive test dependencies.

Exercise 4 - Invocation count: Write a tiny test with invocationCount = 3 and increment/log a counter per invocation. Confirm the report shows repeated invocations.

Exercise 5 - Timeout: Create a deliberately slow test and apply a reasonable timeOut. Observe how TestNG reports a timeout.

Exercise 6 - Enabled flag: Set enabled = false on a test. Run the class and confirm that the disabled method is not executed normally.

Exercise 7 - DataProvider: Provide three search strings through Object[][] and use one @Test method to assert that each value is not null or blank.

Exercise 8 - Lifecycle: Add @BeforeMethod and @AfterMethod messages. Run two tests and count how often setup/teardown execute.

Exercise 9 - Listener: Register ITestListener in testng.xml. Make one passing and one failing assertion and inspect the log callbacks.

Exercise 10 - Selenium cleanup: Launch a test browser in @BeforeMethod and quit it in @AfterMethod. Force an assertion failure and confirm the browser still closes.

Suggested mini-project

Build a small login-and-contacts suite for a test environment. Include one valid-login test, several invalid-login rows through a DataProvider, one contact-creation test, a screenshot-on-failure hook, safe credential handling, and cleanup. Run smoke tests separately from regression tests through groups in testng.xml.

19. Interview revision sheet

Question	Answer
What is TestNG?	A Java test framework for defining, configuring, organizing, executing, and reporting tests.
Does a TestNG test need main()?	No. A test runner such as TestNG invokes @Test methods.
What does @Test do?	Marks a method or class as part of the TestNG test suite.
What is priority?	A numeric scheduling value; lower priorities are scheduled first.
What is the default priority?	0.
Can priority be negative?	Yes.
What does invocationCount do?	Sets how many times a test method is invoked.
What does invocationTimeOut do?	Limits the cumulative time for invocation-count executions; it is ignored when invocationCount is not specified.
What does threadPoolSize do?	Allows repeated invocations to execute using multiple threads when invocationCount is specified.
What does timeOut do?	Sets the maximum time in milliseconds for a test invocation.
How do you skip a test?	Use @Test(enabled = false); @Ignore is another TestNG mechanism with scope depending on where applied.
Why use @DataProvider?	To run the same test logic with multiple sets of input data.
What is @BeforeMethod used for?	Setup before each test-method invocation.
What is @AfterMethod used for?	Cleanup after each test-method invocation.
What is a listener?	A callback interface that observes test lifecycle events for logging/reporting/diagnostics.
What is a common parallel-testing hazard?	Sharing WebDriver or mutable test data across concurrent tests.
Why use assertions?	To verify that observed behavior matches the expected requirement.
What is the main risk with SoftAssert?	Forgetting assertAll() can leave collected failures unreported as test failures.
Should tests depend on execution order?	Generally no. Use independent setup; explicit dependencies are for genuine prerequisites.
Why is Thread.sleep discouraged?	It waits a fixed duration regardless of whether the required condition is ready.

20. Glossary and references

Term	Meaning
Annotation	Java metadata such as @Test that TestNG reads to configure execution.
Test method	A Java method that TestNG executes as a test.
Fixture	Setup or cleanup code that creates or removes conditions needed by a test.
Assertion	A check that compares actual behavior with expected behavior.
Data-driven test	One test method executed with multiple sets of input data.
Flaky test	A test whose result changes unpredictably without a relevant code change.
Thread-safe	Code or state that can be accessed concurrently without unsafe interference.
Test suite	A configured collection of tests and execution settings, often described in testng.xml.

Official references checked during preparation:

- TestNG documentation: https://testng.org/documentation.html
- TestNG releases (version verification): https://github.com/testng-team/testng/releases
- TestNG Maven setup: https://testng.org/maven
- TestNG Eclipse plug-in setup: https://testng.org/testng-eclipse/download.html